

Export de funciones public de Supabase

Proyecto: SmartOps

Project ref: ucuenkacpdibheuyabdb

Fecha de exportacion: 20/4/2026, 12:10:43 p. m.

Total de funciones public exportadas: 35

Inventario

1. can_manage_role_permissions (p_role_id uuid) - RPC / callable
2. change_company_plan (p_company_id uuid, p_plan_key text) - RPC / callable
3. check_registration_availability (p_id_card text, p_rnc text) - RPC / callable
4. configure_payment_daily_reminders_job (sin parametros) - RPC / callable
5. create_installation_project (p_company_id uuid, p_budget_id uuid, p_tecnici
n_id uuid, p_scheduled_start timestamp with time zone, p_scheduled_end timestamp
with time zone) - RPC / callable
6. create_payment_account_from_delivery_act (p_delivery_act_id uuid, p_created_b
y uuid) - RPC / callable
7. enforce_budget_items_editable (sin parametros) - Auxiliar / trigger
8. enforce_budget_layout_lock (sin parametros) - Auxiliar / trigger
9. enforce_budget_status_transition (sin parametros) - Auxiliar / trigger
10. enforce_installed_device_project_consistency (sin parametros) - Auxiliar / t
rigger
11. enforce_user_roles_plan_limits (sin parametros) - Auxiliar / trigger
12. ensure_company_subscription (p_company_id uuid) - Auxiliar / trigger
13. ensure_payment_account_for_project (p_project_id uuid, p_created_by uuid) -
RPC / callable
14. ensure_post_installation_checklist (p_company_id uuid) - RPC / callable
15. finalize_installation_project (p_company_id uuid, p_project_id uuid, p_user_
id uuid) - RPC / callable
16. get_basic_plan_id (sin parametros) - RPC / callable
17. get_company_plan_limits (p_company_id uuid) - RPC / callable
18. get_my_company_id (sin parametros) - RPC / callable
19. get_superuser_company_detail (p_company_id uuid) - RPC / callable
20. get_superuser_overview (sin parametros) - RPC / callable
21. handle_new_auth_user (sin parametros) - Auxiliar / trigger
22. normalize_role_name (role_name text) - Auxiliar / trigger
23. raise_limit_exceeded (p_company_id uuid, p_resource text, p_limit integer, p
_current integer, p_plan_key text) - Auxiliar / trigger
24. refresh_payment_account_balance (p_account_id uuid) - Auxiliar / trigger
25. run_payment_daily_reminders (sin parametros) - RPC / callable
26. seed_installation_project_defaults (p_company_id uuid, p_project_id uuid) -
RPC / callable
27. set_customer_invitations_updated_at (sin parametros) - Auxiliar / trigger
28. set_customer_profile_360_updated_at (sin parametros) - Auxiliar / trigger
29. set_updated_at (sin parametros) - Auxiliar / trigger
30. trg_payment_scheduler_set_updated_at (sin parametros) - Auxiliar / trigger
31. trg_payment_scheduler_sync (sin parametros) - Auxiliar / trigger
32. trg_refresh_payment_account_balance (sin parametros) - Auxiliar / trigger
33. update_checklist_templates_updated_at (sin parametros) - Auxiliar / trigger
34. upsert_installation_project_visit (p_company_id uuid, p_project_id uuid, p_t
echnician_id uuid, p_scheduled_start timestamp with time zone, p_scheduled_end t
imestamp with time zone) - RPC / callable
35. upsert_project_post_installation_check_state (p_company_id uuid, p_project_i
d uuid, p_item_id uuid, p_checked boolean, p_notes text, p_user_id uuid) - RPC /
callable

Codigo SQL por funcion

1. can_manage_role_permissions

- Schema: public

- Parametros: p_role_id uuid
- Clasificacion: RPC / callable
- Archivo separado: database_function_exports/01_can_manage_role_permissions__p_role_id_uuid.sql

```
```sql
CREATE OR REPLACE FUNCTION public.can_manage_role_permissions(p_role_id uuid)
RETURNS boolean
LANGUAGE sql
STABLE SECURITY DEFINER
SET search_path TO 'public'
AS $function$
 select exists (
 select 1
 from public.roles r
 join public.user_roles ur
 on ur.company_id = r.company_id
 where r.id = p_role_id
 and ur.user_id = auth.uid()
);
$function$
```
```

2. change_company_plan

- Schema: public
- Parametros: p_company_id uuid, p_plan_key text
- Clasificacion: RPC / callable
- Archivo separado: database_function_exports/02_change_company_plan__p_company_id_uuid_p_plan_key_text.sql

```
```sql
CREATE OR REPLACE FUNCTION public.change_company_plan(p_company_id uuid, p_plan_key text)
RETURNS void
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
declare
 v_plan_id uuid;
 v_is_allowed boolean;
begin
 if p_company_id is null then
 raise exception 'company_id is required';
 end if;

 if auth.uid() is null then
 raise exception 'No authenticated user';
 end if;

 select id
 into v_plan_id
 from public.pricing_plans
 where key = p_plan_key
 and is_active = true
 limit 1;

 if v_plan_id is null then
 raise exception 'Invalid plan key: %', coalesce(p_plan_key, 'null');
 end if;
end if;
$function$
```
```

```

-- Only allow authenticated members of the company that are not customers.
if to_regclass('public.user_roles') is null or to_regclass('public.roles') is
null then
    raise exception 'Missing required tables (user_roles/roles) to validate perm
issions';
end if;

select exists (
    select 1
    from public.user_roles ur
    join public.roles r on r.id = ur.role_id
    where ur.user_id = auth.uid()
    and ur.company_id = p_company_id
    and public.normalize_role_name(r.name) <> 'customer'
)
into v_is_allowed;

if not v_is_allowed then
    raise exception using
        errcode = '42501',
        message = 'Not allowed to change plan for this company';
end if;

perform public.ensure_company_subscription(p_company_id);

update public.company_subscriptions
set
    plan_id = v_plan_id,
    status = 'active',
    end_date = null,
    updated_at = now()
where company_id = p_company_id;
end;
$function$

```

...

3. check_registration_availability

- Schema: public
- Parametros: p_id_card text, p_rnc text
- Clasificación: RPC / callable
- Archivo separado: database_function_exports/03_check_registration_availability_p_id_card_text_p_rnc_text.sql

```

```sql
CREATE OR REPLACE FUNCTION public.check_registration_availability(p_id_card text
, p_rnc text)
RETURNS json
LANGUAGE plpgsql
SECURITY DEFINER
AS $function$
BEGIN
 IF EXISTS (SELECT 1 FROM public.users WHERE id_card = p_id_card) THEN
 RETURN json_build_object('available', false, 'reason', 'La cédula ya está re
gistrada');
 END IF;

 IF EXISTS (SELECT 1 FROM public.companies WHERE rnc = p_rnc) THEN
 RETURN json_build_object('available', false, 'reason', 'El RNC ya está regis
trado');
 END IF;
END;
$function$

```

```

 END IF;

 RETURN json_build_object('available', true, 'reason', null);
END;
$function$
```

### 4. configure_payment_daily_reminders_job

- Schema: public
- Parametros: sin parametros
- Clasificacion: RPC / callable
- Archivo separado: database_function_exports/04_configure_payment_daily_reminders_job__sin_parametros.sql

```sql
CREATE OR REPLACE FUNCTION public.configure_payment_daily_reminders_job()
 RETURNS void
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public', 'cron'
AS $function$
declare
 v_config public.payment_scheduler_config%rowtype;
 v_job record;
begin
 for v_job in
 select jobid
 from cron.job
 where jobname = 'payment-daily-reminders'
 loop
 perform cron.unschedule(v_job.jobid);
 end loop;

 select *
 into v_config
 from public.payment_scheduler_config
 where id = true;

 if v_config is null or not v_config.enabled then
 return;
 end if;

 perform cron.schedule(
 'payment-daily-reminders',
 v_config.reminders_schedule,
 jobselect public.run_payment_daily_reminders();job
);
end;
$function$
```

```

5. create_installation_project

```

- Schema: public
- Parametros: p_company_id uuid, p_budget_id uuid, p_technician_id uuid, p_scheduled_start timestamp with time zone, p_scheduled_end timestamp with time zone
- Clasificacion: RPC / callable
- Archivo separado: database_function_exports/05_create_installation_project__p_company_id_uuid_p_budget_id_uuid_p_technician_id_uuid_p_scheduled_start_timestamp

```

p_with_time_zone_p_scheduled_end_times.sql

```
```sql
CREATE OR REPLACE FUNCTION public.create_installation_project(p_company_id uuid,
 p_budget_id uuid, p_technician_id uuid DEFAULT NULL::uuid, p_scheduled_start ti
mestamp with time zone DEFAULT NULL::timestamp with time zone, p_scheduled_end t
imestamp with time zone DEFAULT NULL::timestamp with time zone)
 RETURNS TABLE(project_id uuid, technical_visit_id uuid, created boolean)
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public', 'extensions'
AS $function$
declare
 v_budget record;
 v_project_id uuid;
 v_visit_id uuid;
 v_uid uuid;
 v_is_member boolean;
begin
 v_uid := auth.uid();

 if v_uid is null then
 raise exception 'No autenticado';
 end if;

 select exists (
 select 1
 from public.user_roles ur
 where ur.user_id = v_uid
 and ur.company_id = p_company_id
)
 or exists (
 select 1
 from public.user_roles ur
 join public.roles r on r.id = ur.role_id
 where ur.user_id = v_uid
 and ur.company_id is null
 and lower(r.name) = 'admin'
)
 into v_is_member;

 if not coalesce(v_is_member, false) then
 raise exception 'No autorizado para esta compania';
 end if;

 select b.id, b.status, b.survey_id, ss.site_id
 into v_budget
 from public.budgets b
 join public.site_surveys ss on ss.id = b.survey_id
 where b.id = p_budget_id
 and b.company_id = p_company_id;

 if v_budget is null then
 raise exception 'Presupuesto no encontrado';
 end if;

 if lower(coalesce(v_budget.status, '')) <> 'aprobada' then
 raise exception 'No se puede crear OT: la cotizacion no esta aprobada';
 end if;

 select id into v_project_id
 from public.installation_projects
```

```

where budget_id = p_budget_id
limit 1;

if v_project_id is not null then
 select id into v_visit_id
 from public.technical_visits
 where installation_project_id = v_project_id
 order by scheduled_start asc nulls last
 limit 1;

 return query select v_project_id, v_visit_id, false;
return;
end if;

insert into public.installation_projects (
 company_id,
 budget_id,
 site_id,
 responsible_user_id,
 status,
 created_at,
 updated_at
)
values (
 p_company_id,
 p_budget_id,
 v_budget.site_id,
 null,
 'pendiente',
 now(),
 now()
)
returning id into v_project_id;

return query select v_project_id, null::uuid, true;
end;
$function$

```

```

...

6. create_payment_account_from_delivery_act

```

- Schema: public
- Parametros: p\_delivery\_act\_id uuid, p\_created\_by uuid
- Clasificacion: RPC / callable
- Archivo separado: database\_function\_exports/06\_create\_payment\_account\_from\_delivery\_act\_\_p\_delivery\_act\_id\_uuid\_p\_created\_by\_uuid.sql

```

```sql
CREATE OR REPLACE FUNCTION public.create_payment_account_from_delivery_act(p_delivery_act_id uuid, p_created_by uuid DEFAULT NULL::uuid)
RETURNS TABLE(account_id uuid, company_id uuid, customer_id uuid, project_id uuid, budget_id uuid, amount_total numeric, amount_pending numeric, invoice_number text, created boolean)
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
declare
    v_act record;
    v_project record;
    v_budget_total numeric(12, 2);

```

```

v_account public.payment_accounts%rowtype;
v_account_id uuid;
v_invoice_number text;
begin
    if to_regclass('public.delivery_acts') is null then
        raise exception 'La tabla delivery_acts no esta disponible en este entorno.'
    ;
    end if;

    execute
        'select id, company_id, project_id, customer_id from public.delivery_acts wh
ere id = $1'
    into v_act
    using p_delivery_act_id;

    if v_act is null then
        raise exception 'No se encontro el acta indicada.';
    end if;

    select pa.*
    into v_account
    from public.payment_accounts pa
    where pa.project_id = v_act.project_id
    limit 1;

    if found then
        return query
        select
            v_account.id,
            v_account.company_id,
            v_account.customer_id,
            v_account.project_id,
            v_account.budget_id,
            v_account.amount_total,
            v_account.amount_pending,
            v_account.invoice_number,
            false;
        return;
    end if;

    select ip.id, ip.company_id, ip.budget_id
    into v_project
    from public.installation_projects ip
    where ip.id = v_act.project_id
        and ip.company_id = v_act.company_id;

    if v_project is null then
        raise exception 'No se encontro el proyecto relacionado al acta.';
    end if;

    select coalesce(b.total, 0)
    into v_budget_total
    from public.budgets b
    where b.id = v_project.budget_id
        and b.company_id = v_project.company_id;

    v_account_id := gen_random_uuid();
    v_invoice_number := 'FAC-' || to_char(now(), 'YYYYMMDD') || '-' || upper(right
(replace(v_account_id::text, '-', ''), 6));

    insert into public.payment_accounts (
        id,

```

```

company_id,
customer_id,
project_id,
budget_id,
delivery_act_id,
status,
currency,
amount_total,
amount_paid,
amount_pending,
invoice_number,
created_by,
created_at,
updated_at
) values (
v_account_id,
v_project.company_id,
v_act.customer_id,
v_project.id,
v_project.budget_id,
p_delivery_act_id,
'pending',
'USD',
v_budget_total,
0,
v_budget_total,
v_invoice_number,
p_created_by,
now(),
now()
);

return query
select
v_account_id,
v_project.company_id,
v_act.customer_id,
v_project.id,
v_project.budget_id,
v_budget_total,
v_budget_total,
v_invoice_number,
true;

end;
$function$

```

7. enforce_budget_items_editable

- Schema: public
- Parametros: sin parametros
- Clasificacion: Auxiliar / trigger
- Archivo separado: database_function_exports/07_enforce_budget_items_editable__sin_parametros.sql

```sql
CREATE OR REPLACE FUNCTION public.enforce_budget_items_editable()
RETURNS trigger
LANGUAGE plpgsql
AS $function$
declare

```

```

    v_budget_id uuid;
    v_status text;
    v_has_ot boolean;
begin
    v_budget_id := coalesce(new.budget_id, old.budget_id);

    select b.status,
           exists (select 1 from public.installation_projects ip where ip.budget_id = b.id)
    into v_status, v_has_ot
    from public.budgets b
    where b.id = v_budget_id
    limit 1;

    if v_status is null then
        raise exception 'Presupuesto no encontrado para validar items';
    end if;

    if lower(coalesce(v_status, '')) <> 'borrador' then
        raise exception 'No se pueden modificar items cuando el presupuesto no esta en borrador';
    end if;

    if coalesce(v_has_ot, false) then
        raise exception 'No se pueden modificar items: la OT ya fue creada';
    end if;

    if tg_op = 'DELETE' then
        return old;
    end if;

    return new;
end;
$function$

```

8. enforce_budget_layout_lock

- Schema: public
- Parametros: sin parametros
- Clasificacion: Auxiliar / trigger
- Archivo separado: database_function_exports/08_enforce_budget_layout_lock__sin_parametros.sql

```

```sql
CREATE OR REPLACE FUNCTION public.enforce_budget_layout_lock()
RETURNS trigger
LANGUAGE plpgsql
AS $function$
begin
 if (
 new.layout_json is distinct from old.layout_json
 or new.subtotal is distinct from old.subtotal
 or new.tax_rate is distinct from old.tax_rate
 or new.tax_amount is distinct from old.tax_amount
 or new.total is distinct from old.total
) then
 if lower(coalesce(old.status, '')) <> 'borrador' then
 raise exception 'No se puede modificar el plano/totales cuando el presupuesto no esta en borrador';
 end if;
 end if;

```

```

 if exists (
 select 1
 from public.installation_projects ip
 where ip.budget_id = old.id
) then
 raise exception 'No se puede modificar el plano/totales: la OT ya fue creada';
 end if;
 end if;

 return new;
end;
$function$
```

```

9. enforce_budget_status_transition

- Schema: public
- Parametros: sin parametros
- Clasificacion: Auxiliar / trigger
- Archivo separado: database_function_exports/09_enforce_budget_status_transition_sin_parametros.sql

```

```sql
CREATE OR REPLACE FUNCTION public.enforce_budget_status_transition()
RETURNS trigger
LANGUAGE plpgsql
AS $function$
begin
 if new.status is distinct from old.status then
 if new.status in ('aprobada', 'rechazada') and old.status <> 'enviada' then
 raise exception 'Solo se puede aprobar/rechazar una cotizacion enviada';
 end if;
 end if;

 return new;
end;
$function$
```

```

10. enforce_installed_device_project_consistency

- Schema: public
- Parametros: sin parametros
- Clasificacion: Auxiliar / trigger
- Archivo separado: database_function_exports/10_enforce_installed_device_project_consistency_sin_parametros.sql

```

```sql
CREATE OR REPLACE FUNCTION public.enforce_installed_device_project_consistency()
RETURNS trigger
LANGUAGE plpgsql
AS $function$
declare
 proj_company_id uuid;
 proj_site_id uuid;
begin
 select p.company_id, p.site_id
 into proj_company_id, proj_site_id

```

```

from public.installation_projects p
where p.id = new.project_id;

if proj_company_id is null then
 raise exception 'Proyecto no encontrado para project_id=%', new.project_id;
end if;

if new.company_id <> proj_company_id then
 raise exception 'company_id no coincide con el proyecto';
end if;

if new.site_id <> proj_site_id then
 raise exception 'site_id no coincide con el proyecto';
end if;

return new;
end;
$function$
```

### 11. enforce_user_roles_plan_limits

- Schema: public
- Parametros: sin parametros
- Clasificacion: Auxiliar / trigger
- Archivo separado: database_function_exports/11_enforce_user_roles_plan_limits_
_sin_parametros.sql

```sql
CREATE OR REPLACE FUNCTION public.enforce_user_roles_plan_limits()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
declare
 role_norm text;
 limits record;
 current_count integer;
begin
 if new.company_id is null then
 return new;
 end if;

 select public.normalize_role_name(r.name)
 into role_norm
 from public.roles r
 where r.id = new.role_id;

 if role_norm is null then
 return new;
 end if;

 select * into limits
 from public.get_company_plan_limits(new.company_id);

 -- Clientes (rol customer) -> max_clients
 if role_norm = 'customer' then
 if limits.max_clients is null then
 return new;
 end if;
 end if;
end;
$function$

```

```

select count(*) into current_count
from public.user_roles ur
join public.roles r on r.id = ur.role_id
where ur.company_id = new.company_id
and public.normalize_role_name(r.name) = 'customer';

if current_count >= limits.max_clients then
perform public.raise_limit_exceeded(new.company_id, 'clients', limits.max_
clients, current_count, limits.plan_key);
end if;

return new;
end if;

-- Cualquier miembro NO customer cuenta como "usuario interno" -> max_tecnici
ans
if limits.max_technicians is null then
return new;
end if;

select count(*) into current_count
from public.user_roles ur
join public.roles r on r.id = ur.role_id
where ur.company_id = new.company_id
and public.normalize_role_name(r.name) <> 'customer';

if current_count >= limits.max_technicians then
perform public.raise_limit_exceeded(new.company_id, 'technicians', limits.ma
x_technicians, current_count, limits.plan_key);
end if;

return new;
end;
$function$
...

```

### ### 12. ensure\_company\_subscription

- Schema: public
- Parametros: p\_company\_id uuid
- Clasificacion: Auxiliar / trigger
- Archivo separado: database\_function\_exports/12\_ensure\_company\_subscription\_\_p\_company\_id\_uuid.sql

```

```sql
CREATE OR REPLACE FUNCTION public.ensure_company_subscription(p_company_id uuid)
RETURNS void
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
declare
basic_plan_id uuid;
begin
if p_company_id is null then
return;
end if;

if exists (select 1 from public.company_subscriptions cs where cs.company_id =
p_company_id) then

```

```

        return;
    end if;

    basic_plan_id := public.get_basic_plan_id();
    if basic_plan_id is null then
        raise exception 'No existe el plan base (basic) para inicializar suscripcion
es.';
    end if;

    insert into public.company_subscriptions (company_id, plan_id, status, start_d
ate)
    values (p_company_id, basic_plan_id, 'active', now())
    on conflict (company_id) do nothing;
end;
$function$
```

```

### ### 13. ensure\_payment\_account\_for\_project

- Schema: public
- Parametros: p\_project\_id uuid, p\_created\_by uuid
- Clasificacion: RPC / callable
- Archivo separado: database\_function\_exports/13\_ensure\_payment\_account\_for\_proj
ect\_\_p\_project\_id\_uuid\_p\_created\_by\_uuid.sql

```

```sql
CREATE OR REPLACE FUNCTION public.ensure_payment_account_for_project(p_project_i
d uuid, p_created_by uuid DEFAULT NULL::uuid)
RETURNS TABLE(account_id uuid, delivery_act_id uuid, created boolean)
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
declare
    v_act record;
    v_created record;
begin
    select da.id, da.created_by, da.signed_by
    into v_act
    from public.delivery_acts da
    where da.project_id = p_project_id
        and da.status in ('signed', 'accepted')
    order by coalesce(da.signed_at, da.accepted_at, da.updated_at, da.created_at)
desc
    limit 1;

    if v_act is null then
        raise exception 'No existe un acta firmada/aceptada para este proyecto.';
    end if;

    select created_account.account_id, created_account.created
    into v_created
    from public.create_payment_account_from_delivery_act(
        v_act.id,
        coalesce(p_created_by, v_act.signed_by, v_act.created_by)
    ) as created_account;

    return query
    select
        v_created.account_id,
        v_act.id,

```

```

        v_created.created;
end;
$function$
```

```

### ### 14. ensure\_post\_installation\_checklist

- Schema: public
- Parametros: p\_company\_id uuid
- Clasificacion: RPC / callable
- Archivo separado: database\_function\_exports/14\_ensure\_post\_installation\_checklist\_\_p\_company\_id\_uuid.sql

```

```sql
CREATE OR REPLACE FUNCTION public.ensure_post_installation_checklist(p_company_id uuid)
RETURNS uuid
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public', 'extensions'
AS $function$
declare
    v_uid uuid;
    v_is_member boolean;
    v_checklist_id uuid;
begin
    v_uid := auth.uid();
    if v_uid is null then
        raise exception 'No autenticado';
    end if;

    select exists (
        select 1
        from public.user_roles ur
        where ur.user_id = v_uid
        and ur.company_id = p_company_id
    )
    or exists (
        select 1
        from public.user_roles ur
        join public.roles r on r.id = ur.role_id
        where ur.user_id = v_uid
        and ur.company_id is null
        and lower(r.name) = 'admin'
    )
    into v_is_member;

    if not coalesce(v_is_member, false) then
        raise exception 'No autorizado para esta compania';
    end if;

    insert into public.post_installation_checks (company_id, name, description)
    values (p_company_id, 'Pruebas post instalacion', 'Checklist general para cerrar proyectos de instalacion')
    on conflict (company_id)
    do update set updated_at = now()
    returning id into v_checklist_id;

    insert into public.post_installation_check_items (checklist_id, text, item_order)
    values

```

```

(v_checklist_id, 'Encendido general verificado', 0),
(v_checklist_id, 'Comunicacion estable entre dispositivos', 1),
(v_checklist_id, 'Automatizaciones principales funcionando', 2),
(v_checklist_id, 'Respuesta en app/control confirmada', 3),
(v_checklist_id, 'Capacitacion basica al cliente completada', 4)
on conflict (checklist_id, text) do nothing;

return v_checklist_id;
end;
$function$
...

### 15. finalize_installation_project

- Schema: public
- Parametros: p_company_id uuid, p_project_id uuid, p_user_id uuid
- Clasificacion: RPC / callable
- Archivo separado: database_function_exports/15_finalize_installation_project__
p_company_id_uuid_p_project_id_uuid_p_user_id_uuid.sql

```sql
CREATE OR REPLACE FUNCTION public.finalize_installation_project(p_company_id uui
d, p_project_id uuid, p_user_id uuid DEFAULT NULL::uuid)
RETURNS TABLE(project_id uuid, already_finalized boolean, inventory_consumed bo
olean)
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public', 'extensions'
AS $function$
declare
v_uid uuid;
v_actor uuid;
v_is_member boolean;
v_project record;
v_visit record;
v_phases_total integer;
v_phases_pending integer;
v_post_total integer;
v_post_pending integer;
v_inventory record;
v_req record;
v_consumed boolean := false;
v_idempotency_key text := 'finalize_v1';
begin
v_uid := auth.uid();

if v_uid is null then
raise exception 'No autenticado';
end if;

v_actor := coalesce(p_user_id, v_uid);

select exists (
select 1
from public.user_roles ur
where ur.user_id = v_uid
and ur.company_id = p_company_id
)
or exists (
select 1
from public.user_roles ur

```

```

 join public.roles r on r.id = ur.role_id
 where ur.user_id = v_uid
 and ur.company_id is null
 and lower(r.name) = 'admin'
)
into v_is_member;

if not coalesce(v_is_member, false) then
 raise exception 'No autorizado para esta compania';
end if;

select ip.id, ip.budget_id, ip.status, ip.phases, ip.post_installation_checks_
state
into v_project
from public.installation_projects ip
where ip.id = p_project_id
 and ip.company_id = p_company_id
for update;

if v_project is null then
 raise exception 'Proyecto no encontrado';
end if;

if lower(coalesce(v_project.status, '')) = 'cancelado' then
 raise exception 'No se puede cerrar un proyecto cancelado';
end if;

if lower(coalesce(v_project.status, '')) = 'terminado' then
 return query select p_project_id, true, false;
 return;
end if;

select tv.id, tv.status
into v_visit
from public.technical_visits tv
where tv.installation_project_id = p_project_id
order by tv.created_at desc nulls last, tv.scheduled_start desc nulls last
limit 1;

if v_visit is null then
 raise exception 'Debes programar una visita tecnica antes de cerrar el proye
cto';
end if;

if lower(coalesce(v_visit.status, '')) = 'cancelada' then
 raise exception 'La visita tecnica del proyecto esta cancelada. Reprograma p
rimero';
end if;

select
 count(*)::integer,
 count(*) filter (
 where (
 case
 when jsonb_typeof(phase_item.value) = 'object'
 then coalesce((phase_item.value ->> 'done')::boolean, false) = false
 when jsonb_typeof(phase_item.value) = 'string'
 then true
 else true
 end
)
)
and (

```

```

 case
 when jsonb_typeof(phase_item.value) = 'object'
 then length(btrim(coalesce(phase_item.value ->> 'title', ''))) > 0
 when jsonb_typeof(phase_item.value) = 'string'
 then length(btrim(trim(both '' from phase_item.value::text))) > 0
 else false
 end
)
)::integer
into v_phases_total, v_phases_pending
from jsonb_array_elements(coalesce(v_project.phases, '[]'::jsonb)) as phase_it
em(value)
where (
 case
 when jsonb_typeof(phase_item.value) = 'object'
 then length(btrim(coalesce(phase_item.value ->> 'title', ''))) > 0
 when jsonb_typeof(phase_item.value) = 'string'
 then length(btrim(trim(both '' from phase_item.value::text))) > 0
 else false
 end
);

if coalesce(v_phases_total, 0) = 0 then
 raise exception 'Debes definir al menos una fase en el plan del proyecto ant
es de cerrar';
end if;

if coalesce(v_phases_pending, 0) > 0 then
 raise exception 'No puedes cerrar: faltan fases del plan por completar';
end if;

select
 count(*)::integer,
 count(*) filter (
 where coalesce(
 (coalesce(v_project.post_installation_checks_state, '{}'::jsonb) -> (pci
.id::text) ->> 'checked')::boolean,
 false
) = false
)::integer
into v_post_total, v_post_pending
from public.post_installation_checks pc
join public.post_installation_check_items pci on pci.checklist_id = pc.id
where pc.company_id = p_company_id
 and pci.is_active = true;

if coalesce(v_post_total, 0) = 0 then
 raise exception 'Debes configurar los items de pruebas post instalacion ante
s de cerrar';
end if;

if coalesce(v_post_pending, 0) > 0 then
 raise exception 'No puedes cerrar: faltan items en pruebas post instalacion'
;
end if;

if exists (
 select 1
 from public.installation_project_inventory_consumption c
 where c.company_id = p_company_id
 and c.project_id = p_project_id
 and c.idempotency_key = v_idempotency_key

```

```

limit 1
) then
v_consumed := false;
else
for v_req in
select
bi.id as budget_item_id,
bi.device_id,
greatest(coalesce(bi.quantity, 1), 1)::integer as required_qty
from public.installation_projects ip
join public.budget_items bi on bi.budget_id = ip.budget_id
where ip.id = p_project_id
order by bi.created_at asc, bi.id asc
loop
select di.id, di.quantity
into v_inventory
from public.device_inventory di
where di.device_id = v_req.device_id
for update;

if v_inventory.id is null then
raise exception 'Inventario no encontrado para el dispositivo requerido'
;
end if;

if coalesce(v_inventory.quantity, 0) < v_req.required_qty then
raise exception 'Inventario insuficiente para completar la instalacion';
end if;

update public.device_inventory
set quantity = quantity - v_req.required_qty,
status = case
when (quantity - v_req.required_qty) <= 0 then 'out_of_stock'
when (quantity - v_req.required_qty) <= 5 then 'low_stock'
else 'available'
end,
last_updated = now()
where id = v_inventory.id;

insert into public.installation_project_inventory_consumption (
company_id,
project_id,
device_id,
budget_item_id,
idempotency_key,
quantity,
consumed_by,
consumed_at
)
values (
p_company_id,
p_project_id,
v_req.device_id,
v_req.budget_item_id,
v_idempotency_key,
v_req.required_qty,
v_actor,
now()
)
on conflict do nothing;
end loop;

```

```

 v_consumed := true;
 end if;

 update public.installation_projects
 set status = 'terminado',
 completed_at = coalesce(completed_at, now()),
 updated_at = now()
 where id = p_project_id;

 update public.technical_visits
 set status = 'completada',
 scheduled_end = coalesce(scheduled_end, now())
 where installation_project_id = p_project_id
 and status in ('programada', 'en_progreso');

 return query select p_project_id, false, v_consumed;
end;
$function$

```

...

### ### 16. get\_basic\_plan\_id

- Schema: public
- Parametros: sin parametros
- Clasificacion: RPC / callable
- Archivo separado: database\_function\_exports/16\_get\_basic\_plan\_id\_\_sin\_parametros.sql

```

```sql
CREATE OR REPLACE FUNCTION public.get_basic_plan_id()
    RETURNS uuid
    LANGUAGE sql
    STABLE SECURITY DEFINER
    SET search_path TO 'public'
AS $function$
    select id
    from public.pricing_plans
    where key = 'basic'
    order by is_active desc, sort_order asc
    limit 1;
$function$

```

...

17. get_company_plan_limits

- Schema: public
- Parametros: p_company_id uuid
- Clasificacion: RPC / callable
- Archivo separado: database_function_exports/17_get_company_plan_limits__p_company_id_uuid.sql

```

```sql
CREATE OR REPLACE FUNCTION public.get_company_plan_limits(p_company_id uuid)
 RETURNS TABLE(plan_key text, plan_name text, max_clients integer, max_sites integer, max_devices integer, max_technicians integer, max_tickets_per_month integer, allow_simulator boolean, allow_advanced_automations boolean)
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$

```

```

declare
 basic_plan_id uuid;
begin
 perform public.ensure_company_subscription(p_company_id);

 return query
 select
 p.key,
 p.name,
 l.max_clients,
 l.max_sites,
 l.max_devices,
 l.max_technicians,
 l.max_tickets_per_month,
 coalesce(l.allow_simulator, false),
 coalesce(l.allow_advanced_automations, false)
 from public.company_subscriptions cs
 join public.pricing_plans p on p.id = cs.plan_id
 left join public.plan_limits l on l.plan_id = p.id
 where cs.company_id = p_company_id
 and cs.status in ('active', 'trialing', 'past_due')
 order by
 case cs.status
 when 'active' then 1
 when 'trialing' then 2
 when 'past_due' then 3
 else 4
 end,
 cs.start_date desc
 limit 1;

 if not found then
 basic_plan_id := public.get_basic_plan_id();
 if basic_plan_id is null then
 return;
 end if;

 return query
 select
 p.key,
 p.name,
 l.max_clients,
 l.max_sites,
 l.max_devices,
 l.max_technicians,
 l.max_tickets_per_month,
 coalesce(l.allow_simulator, false),
 coalesce(l.allow_advanced_automations, false)
 from public.pricing_plans p
 left join public.plan_limits l on l.plan_id = p.id
 where p.id = basic_plan_id
 limit 1;
 end if;
end;
$function$

```

### 18. get\_my\_company\_id

- Schema: public
- Parametros: sin parametros

- Clasificación: RPC / callable
- Archivo separado: database\_function\_exports/18\_get\_my\_company\_id\_\_sin\_parametros.sql

```
```sql
CREATE OR REPLACE FUNCTION public.get_my_company_id()
  RETURNS uuid
  LANGUAGE sql
  STABLE SECURITY DEFINER
  SET search_path TO 'public'
AS $function$
  SELECT company_id FROM public.user_roles
  WHERE user_id = auth.uid()
  LIMIT 1;
$function$
```
```

### ### 19. get\_superadmin\_company\_detail

- Schema: public
- Parametros: p\_company\_id uuid
- Clasificación: RPC / callable
- Archivo separado: database\_function\_exports/19\_get\_superadmin\_company\_detail\_\_p\_company\_id\_uuid.sql

```
```sql
CREATE OR REPLACE FUNCTION public.get_superadmin_company_detail(p_company_id uuid)
  RETURNS jsonb
  LANGUAGE plpgsql
  SECURITY DEFINER
  SET search_path TO 'public', 'auth'
AS $function$
declare
  v_caller_user_id uuid := auth.uid();
  v_result jsonb;
begin
  if v_caller_user_id is null then
    raise exception 'Debes iniciar sesion para consultar este modulo.';
  end if;

  if not exists (
    select 1
    from public.user_roles ur
    join public.roles r on r.id = ur.role_id
    where ur.user_id = v_caller_user_id
    and lower(regexp_replace(coalesce(r.name, ''), '[\s_-]+', '', 'g')) = 'superadmin'
  ) then
    raise exception 'Solo un superadmin puede consultar este modulo.';
  end if;

  with company_members as (
    select distinct on (ur.company_id, u.id)
      ur.company_id,
      u.id,
      u.name,
      u.id_card,
      u.photo_url,
      au.email
    from public.user_roles ur

```

```

join public.users u on u.id = ur.user_id
left join auth.users au on au.id = u.id
where ur.company_id = p_company_id
order by ur.company_id, u.id
)
select jsonb_build_object(
    'id', c.id,
    'name', c.name,
    'logoUrl', c.logo_url,
    'createdAt', c.created_at,
    'users',
    coalesce(
        (
            select jsonb_agg(
                jsonb_build_object(
                    'id', cm.id,
                    'name', cm.name,
                    'idCard', cm.id_card,
                    'email', cm.email,
                    'photoUrl', cm.photo_url
                )
            )
            order by lower(cm.name), cm.id
        )
        from company_members cm
    ), [] '::jsonb
), 'totalUsers',
(select count(*) from company_members)
)
into v_result
from public.companies c
where c.id = p_company_id;

if v_result is null then
    raise exception 'La compania solicitada no existe.';
end if;

return v_result;
end;
$function$

```

...

20. get_superadmin_overview

- Schema: public
- Parametros: sin parametros
- Clasificacion: RPC / callable
- Archivo separado: database_function_exports/20_get_superadmin_overview__sin_parametros.sql

```

```sql
CREATE OR REPLACE FUNCTION public.get_superadmin_overview()
RETURNS jsonb
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public', 'auth'
AS $function$
declare
 v_caller_user_id uuid := auth.uid();
 v_result jsonb;

```

```

begin
 if v_caller_user_id is null then
 raise exception 'Debes iniciar sesion para consultar este modulo.';
 end if;

 if not exists (
 select 1
 from public.user_roles ur
 join public.roles r on r.id = ur.role_id
 where ur.user_id = v_caller_user_id
 and lower(regex_replace(coalesce(r.name, ''), '\s_-' + '', 'g')) = 'superadmin'
) then
 raise exception 'Solo un superadmin puede consultar este modulo.';
 end if;

 with company_members as (
 select distinct on (ur.company_id, u.id)
 ur.company_id,
 u.id,
 u.name,
 u.id_card,
 u.photo_url,
 au.email
 from public.user_roles ur
 join public.users u on u.id = ur.user_id
 left join auth.users au on au.id = u.id
 where ur.company_id is not null
 order by ur.company_id, u.id
),
 company_payload as (
 select
 c.id,
 c.name,
 c.logo_url,
 c.created_at,
 coalesce(
 (
 select jsonb_agg(
 jsonb_build_object(
 'id', cm.id,
 'name', cm.name,
 'idCard', cm.id_card,
 'email', cm.email,
 'photoUrl', cm.photo_url
)
)
)
)
 from public.companies c
 left join company_members cm on cm.company_id = c.id
 where cm.company_id is not null
),
 users as (
 select count(*)
 from company_members cm
 where cm.company_id = c.id
),
 total_users as (
 select count(*)
 from company_members cm
 where cm.company_id = c.id
)
 select jsonb_build_object(
 'companiesCount',

```

```

count(*),
'totalUsers',
coalesce((select count(distinct cm.id) from company_members cm), 0),
'companies',
coalesce(
 jsonb_agg(
 jsonb_build_object(
 'id', cp.id,
 'name', cp.name,
 'logoUrl', cp.logo_url,
 'createdAt', cp.created_at,
 'users', cp.users,
 'totalUsers', cp.total_users
)
) order by cp.created_at desc, cp.name asc
), [] '::jsonb
)
)
into v_result
from company_payload cp;

return coalesce(
 v_result,
 jsonb_build_object(
 'companiesCount', 0,
 'totalUsers', 0,
 'companies', '[]' '::jsonb
)
);
end;
$function$
...

```

### ### 21. handle\_new\_auth\_user

- Schema: public
- Parametros: sin parametros
- Clasificacion: Auxiliar / trigger
- Archivo separado: database\_function\_exports/21\_handle\_new\_auth\_user\_\_sin\_parametros.sql

```

```sql
CREATE OR REPLACE FUNCTION public.handle_new_auth_user()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
AS $function$DECLARE
    v_company_id uuid;
    v_admin_role_id uuid;
    v_selected_role_id uuid;
    v_role_text text;
    v_company_id_text text;
BEGIN
    -- Solo cuando pasa de NO confirmado -> confirmado
    IF NOT (
        OLD.email_confirmed_at IS NULL
        AND NEW.email_confirmed_at IS NOT NULL
    ) THEN
        RETURN NEW;
    END IF;

```

```

-- Si es invitado de cliente, no ejecutar logica de users/companies/user_roles
IF COALESCE(NEW.raw_user_meta_data->>'invited_customer', 'false') = 'true' THE
N
    RETURN NEW;
END IF;

-- 1) Perfil usuario (idempotente)
INSERT INTO public.users (id, name, id_card)
VALUES (
    NEW.id,
    COALESCE(NEW.raw_user_meta_data->>'name', ''),
    NULLIF(NEW.raw_user_meta_data->>'idCard', '')
)
ON CONFLICT (id) DO UPDATE
SET
    name = EXCLUDED.name,
    id_card = EXCLUDED.id_card;

-- 2) Resolver company_id
v_company_id_text := NEW.raw_user_meta_data->>'company_id';

IF v_company_id_text IS NOT NULL
    AND v_company_id_text ~* '^[0-9a-f]{8}-[0-9a-f]{4}-[1-5][0-9a-f]{3}-[89ab][
0-9a-f]{3}-[0-9a-f]{12}$'
THEN
    v_company_id := v_company_id_text::uuid;

    IF NOT EXISTS (
        SELECT 1
        FROM public.companies c
        WHERE c.id = v_company_id
    ) THEN
        v_company_id := NULL;
    END IF;
END IF;

IF v_company_id IS NULL
    AND COALESCE(NEW.raw_user_meta_data->>'company_name', '') <> ''
THEN
    INSERT INTO public.companies (name, address, phone, rnc, created_by)
    VALUES (
        NEW.raw_user_meta_data->>'company_name',
        NULLIF(NEW.raw_user_meta_data->>'company_address', ''),
        NULLIF(NEW.raw_user_meta_data->>'company_phone', ''),
        NULLIF(NEW.raw_user_meta_data->>'company_rnc', ''),
        NEW.id
    )
    RETURNING id INTO v_company_id;
END IF;

-- 3) Resolver rol
SELECT r.id
    INTO v_admin_role_id
FROM public.roles r
WHERE LOWER(r.name) = 'admin'
LIMIT 1;

v_selected_role_id := v_admin_role_id;
v_role_text := NEW.raw_user_meta_data->>'role_id';

IF v_role_text IS NOT NULL

```

```

        AND v_role_text ~* '^[0-9a-f]{8}-[0-9a-f]{4}-[1-5][0-9a-f]{3}-[89ab][0-9a-f]{3}-[0-9a-f]{12}$'
    THEN
        SELECT r.id
            INTO v_selected_role_id
        FROM public.roles r
        WHERE r.id = v_role_text::uuid
        LIMIT 1;

        IF v_selected_role_id IS NULL THEN
            v_selected_role_id := v_admin_role_id;
        END IF;
    END IF;

-- 4) user_roles solo si hay company y role
IF v_company_id IS NOT NULL AND v_selected_role_id IS NOT NULL THEN
    -- Primero intenta update (evita requerir unique constraint para ON CONFLICT
)
    UPDATE public.user_roles
        SET role_id = v_selected_role_id
    WHERE user_id = NEW.id
        AND company_id = v_company_id;

    -- Si no existia fila, inserta
    IF NOT FOUND THEN
        INSERT INTO public.user_roles (user_id, role_id, company_id)
        VALUES (NEW.id, v_selected_role_id, v_company_id);
    END IF;
END IF;

RETURN NEW;

EXCEPTION WHEN OTHERS THEN
    RAISE EXCEPTION 'Error al confirmar usuario %: %', NEW.id, SQLERRM;
END;$function$

```

22. normalize_role_name

- Schema: public
- Parametros: role_name text
- Clasificacion: Auxiliar / trigger
- Archivo separado: database_function_exports/22_normalize_role_name__role_name_text.sql

```

```sql
CREATE OR REPLACE FUNCTION public.normalize_role_name(role_name text)
 RETURNS text
 LANGUAGE sql
 IMMUTABLE
AS $function$
 select regexp_replace(lower(coalesce(role_name, '')), '[\s_-]+', '', 'g');
$function$

```

### ### 23. raise\_limit\_exceeded

- Schema: public
- Parametros: p\_company\_id uuid, p\_resource text, p\_limit integer, p\_current integer, p\_plan\_key text

- Clasificación: Auxiliar / trigger
- Archivo separado: database\_function\_exports/23\_raise\_limit\_exceeded\_\_p\_company\_id\_uuid\_p\_resource\_text\_p\_limit\_integer\_p\_current\_integer\_p\_plan\_key\_text.sql

```
```sql
CREATE OR REPLACE FUNCTION public.raise_limit_exceeded(p_company_id uuid, p_resource text, p_limit integer, p_current integer, p_plan_key text)
RETURNS void
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
begin
    raise exception using
        errcode = 'P0001',
        message = json_build_object(
            'error', 'limit_exceeded',
            'resource', coalesce(p_resource, 'unknown'),
            'companyId', p_company_id,
            'limit', p_limit,
            'current', p_current,
            'planKey', coalesce(p_plan_key, 'unknown')
       )::text;
end;
$function$
```
```

### ### 24. refresh\_payment\_account\_balance

- Schema: public
- Parametros: p\_account\_id uuid
- Clasificación: Auxiliar / trigger
- Archivo separado: database\_function\_exports/24\_refresh\_payment\_account\_balance\_\_p\_account\_id\_uuid.sql

```
```sql
CREATE OR REPLACE FUNCTION public.refresh_payment_account_balance(p_account_id uuid)
RETURNS TABLE(account_id uuid, status text, amount_paid numeric, amount_pending numeric, closed_at timestamp with time zone)
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public'
AS $function$
declare
    v_total numeric(12, 2);
    v_paid numeric(12, 2);
    v_current_status text;
    v_account_id uuid;
    v_status text;
    v_amount_paid numeric(12, 2);
    v_amount_pending numeric(12, 2);
    v_closed_at timestampz;
begin
    select pa.amount_total, pa.status
    into v_total, v_current_status
    from public.payment_accounts pa
    where pa.id = p_account_id;

    if not found then
        return;
    end if;
end;
$function$
```
```

```

end if;

select coalesce(sum(pt.amount), 0)
into v_paid
from public.payment_transactions pt
where pt.payment_account_id = p_account_id
and pt.status = 'approved';

update public.payment_accounts pa
set amount_paid = v_paid,
 amount_pending = greatest(v_total - v_paid, 0),
 status = case
 when v_current_status = 'cancelled' then 'cancelled'
 when greatest(v_total - v_paid, 0) <= 0 then 'paid'
 when v_paid > 0 then 'partial'
 else 'pending'
 end,
 closed_at = case
 when v_current_status = 'cancelled' then pa.closed_at
 when greatest(v_total - v_paid, 0) <= 0 then coalesce(pa.closed_at, now(
))
 else null
 end,
 updated_at = now()
where pa.id = p_account_id
returning pa.id, pa.status, pa.amount_paid, pa.amount_pending, pa.closed_at
into v_account_id, v_status, v_amount_paid, v_amount_pending, v_closed_at;

account_id := v_account_id;
status := v_status;
amount_paid := v_amount_paid;
amount_pending := v_amount_pending;
closed_at := v_closed_at;

return next;
end;
$function$

```

...

### ### 25. run\_payment\_daily\_reminders

- Schema: public
- Parametros: sin parametros
- Clasificacion: RPC / callable
- Archivo separado: database\_function\_exports/25\_run\_payment\_daily\_reminders\_\_sin\_parametros.sql

```

```sql
CREATE OR REPLACE FUNCTION public.run_payment_daily_reminders()
RETURNS bigint
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public', 'extensions'
AS $function$
declare
    v_config public.payment_scheduler_config%rowtype;
    v_request_id bigint;
begin
    select *
    into v_config
    from public.payment_scheduler_config

```

```

where id = true;

if v_config is null or not v_config.enabled then
    return null;
end if;

select net.http_post(
    url := v_config.function_url,
    headers := jsonb_build_object(
        'Content-Type', 'application/json',
        'Authorization', 'Bearer scheduled-task',
        'x-cron-secret', v_config.cron_secret
    ),
    body := jsonb_build_object(
        'mode', 'send_daily_reminders'
    )
)
into v_request_id;

return v_request_id;
end;
$function$
...

### 26. seed_installation_project_defaults

- Schema: public
- Parametros: p_company_id uuid, p_project_id uuid
- Clasificacion: RPC / callable
- Archivo separado: database_function_exports/26_seed_installation_project_defaults__p_company_id_uuid_p_project_id_uuid.sql

```sql
CREATE OR REPLACE FUNCTION public.seed_installation_project_defaults(p_company_id uuid, p_project_id uuid)
RETURNS void
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public', 'extensions'
AS $function$
declare
 v_uid uuid;
 v_is_member boolean;
 v_project record;
 v_checklist_id uuid;
begin
 v_uid := auth.uid();

 if v_uid is null then
 raise exception 'No autenticado';
 end if;

 select exists (
 select 1
 from public.user_roles ur
 where ur.user_id = v_uid
 and ur.company_id = p_company_id
)
 or exists (
 select 1
 from public.user_roles ur

```

```

join public.roles r on r.id = ur.role_id
where ur.user_id = v_uid
 and ur.company_id is null
 and lower(r.name) = 'admin'
)
into v_is_member;

if not coalesce(v_is_member, false) then
 raise exception 'No autorizado para esta compania';
end if;

select ip.id, ip.company_id, ip.budget_id
into v_project
from public.installation_projects ip
where ip.id = p_project_id
 and ip.company_id = p_company_id
limit 1;

if v_project is null then
 raise exception 'Proyecto no encontrado';
end if;

insert into public.installation_project_tasks (
 company_id,
 project_id,
 code,
 title,
 position,
 status
)
values
 (p_company_id, p_project_id, 'cableado', 'Cableado', 0, 'pendiente'),
 (p_company_id, p_project_id, 'configuracion_hub', 'Configuracion hub', 1, 'pendiente'),
 (p_company_id, p_project_id, 'emparejamiento', 'Emparejamiento', 2, 'pendiente'),
 (p_company_id, p_project_id, 'pruebas', 'Pruebas', 3, 'pendiente'),
 (p_company_id, p_project_id, 'capacitacion', 'Capacitacion', 4, 'pendiente')
on conflict (project_id, code)
where code is not null
do nothing;

v_checklist_id := public.ensure_post_installation_checklist(p_company_id);

update public.installation_projects ip
set post_installation_checks_state = (
 select coalesce(
 jsonb_object_agg(
 pci.id::text,
 coalesce(
 ip.post_installation_checks_state -> (pci.id::text),
 jsonb_build_object(
 'checked', false,
 'notes', null,
 'checked_at', null,
 'checked_by', null
)
)
)
)::jsonb
)
from public.post_installation_check_items pci

```

```

 where pci.checklist_id = v_checklist_id
 and pci.is_active = true
),
 updated_at = now()
 where ip.id = p_project_id
 and ip.company_id = p_company_id;
end;
$function$
```

```

27. set_customer_invitations_updated_at

- Schema: public
- Parametros: sin parametros
- Clasificacion: Auxiliar / trigger
- Archivo separado: database_function_exports/27_set_customer_invitations_update_d_at__sin_parametros.sql

```

```sql
CREATE OR REPLACE FUNCTION public.set_customer_invitations_updated_at()
 RETURNS trigger
 LANGUAGE plpgsql
 AS $function$
begin
 new.updated_at = now();
 return new;
end;
$function$
```

```

28. set_customer_profile_360_updated_at

- Schema: public
- Parametros: sin parametros
- Clasificacion: Auxiliar / trigger
- Archivo separado: database_function_exports/28_set_customer_profile_360_update_d_at__sin_parametros.sql

```

```sql
CREATE OR REPLACE FUNCTION public.set_customer_profile_360_updated_at()
 RETURNS trigger
 LANGUAGE plpgsql
 AS $function$
begin
 new.updated_at = now();
 return new;
end;
$function$
```

```

29. set_updated_at

- Schema: public
- Parametros: sin parametros
- Clasificacion: Auxiliar / trigger
- Archivo separado: database_function_exports/29_set_updated_at__sin_parametros.sql

```

```sql

```

```

CREATE OR REPLACE FUNCTION public.set_updated_at()
 RETURNS trigger
 LANGUAGE plpgsql
AS $function$
begin
 new.updated_at = now();
 return new;
end;
$function$

```

### 30. trg\_payment\_scheduler\_set\_updated\_at

- Schema: public
- Parametros: sin parametros
- Clasificacion: Auxiliar / trigger
- Archivo separado: database\_function\_exports/30\_trg\_payment\_scheduler\_set\_updated\_at\_\_sin\_parametros.sql

```

```sql
CREATE OR REPLACE FUNCTION public.trg_payment_scheduler_set_updated_at()
  RETURNS trigger
  LANGUAGE plpgsql
  SECURITY DEFINER
  SET search_path TO 'public'
AS $function$
begin
    new.updated_at = now();
    return new;
end;
$function$

```

31. trg_payment_scheduler_sync

- Schema: public
- Parametros: sin parametros
- Clasificacion: Auxiliar / trigger
- Archivo separado: database_function_exports/31_trg_payment_scheduler_sync__sin_parametros.sql

```

```sql
CREATE OR REPLACE FUNCTION public.trg_payment_scheduler_sync()
 RETURNS trigger
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
begin
 perform public.configure_payment_daily_reminders_job();
 return new;
end;
$function$

```

### 32. trg\_refresh\_payment\_account\_balance

- Schema: public
- Parametros: sin parametros

- Clasificacion: Auxiliar / trigger
- Archivo separado: database\_function\_exports/32\_trg\_refresh\_payment\_account\_balance\_\_sin\_parametros.sql

```
```sql
CREATE OR REPLACE FUNCTION public.trg_refresh_payment_account_balance()
  RETURNS trigger
  LANGUAGE plpgsql
  SECURITY DEFINER
  SET search_path TO 'public'
AS $function$
begin
    perform public.refresh_payment_account_balance(coalesce(new.payment_account_id
, old.payment_account_id));
    return coalesce(new, old);
end;
$function$
```
```

### ### 33. update\_checklist\_templates\_updated\_at

- Schema: public
- Parametros: sin parametros
- Clasificacion: Auxiliar / trigger
- Archivo separado: database\_function\_exports/33\_update\_checklist\_templates\_updated\_at\_\_sin\_parametros.sql

```
```sql
CREATE OR REPLACE FUNCTION public.update_checklist_templates_updated_at()
  RETURNS trigger
  LANGUAGE plpgsql
AS $function$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$function$
```
```

### ### 34. upsert\_installation\_project\_visit

- Schema: public
- Parametros: p\_company\_id uuid, p\_project\_id uuid, p\_technician\_id uuid, p\_scheduled\_start timestamp with time zone, p\_scheduled\_end timestamp with time zone
- Clasificacion: RPC / callable
- Archivo separado: database\_function\_exports/34\_upsert\_installation\_project\_visit\_\_p\_company\_id\_uuid\_p\_project\_id\_uuid\_p\_technician\_id\_uuid\_p\_scheduled\_start\_timestamp\_with\_time\_zone\_p\_scheduled\_end\_time.sql

```
```sql
CREATE OR REPLACE FUNCTION public.upsert_installation_project_visit(p_company_id
  uuid, p_project_id uuid, p_technician_id uuid, p_scheduled_start timestamp with
  time zone, p_scheduled_end timestamp with time zone DEFAULT NULL::timestamp wit
  h time zone)
  RETURNS TABLE(technical_visit_id uuid, created boolean, visit_status text)
  LANGUAGE plpgsql
  SECURITY DEFINER
  SET search_path TO 'public', 'extensions'
AS $function$
declare
```

```

v_uid uuid;
v_is_member boolean;
v_project record;
v_existing_visit record;
v_visit_id uuid;
v_created boolean := false;
begin
    v_uid := auth.uid();

    if v_uid is null then
        raise exception 'No autenticado';
    end if;

    if p_scheduled_end is not null and p_scheduled_end <= p_scheduled_start then
        raise exception 'La fecha/hora fin debe ser mayor que inicio';
    end if;

    select exists (
        select 1
        from public.user_roles ur
        where ur.user_id = v_uid
            and ur.company_id = p_company_id
    )
    or exists (
        select 1
        from public.user_roles ur
        join public.roles r on r.id = ur.role_id
        where ur.user_id = v_uid
            and ur.company_id is null
            and lower(r.name) = 'admin'
    )
    into v_is_member;

    if not coalesce(v_is_member, false) then
        raise exception 'No autorizado para esta compania';
    end if;

    select ip.id, ip.status, b.survey_id
    into v_project
    from public.installation_projects ip
    join public.budgets b on b.id = ip.budget_id
    where ip.id = p_project_id
        and ip.company_id = p_company_id
    limit 1;

    if v_project is null then
        raise exception 'Proyecto no encontrado';
    end if;

    if lower(coalesce(v_project.status, '')) in ('terminado', 'cancelado') then
        raise exception 'No se puede programar visita para un proyecto terminado/cancelado';
    end if;

    select tv.id, tv.status
    into v_existing_visit
    from public.technical_visits tv
    where tv.installation_project_id = p_project_id
    order by tv.created_at desc nulls last, tv.scheduled_start desc nulls last
    limit 1;

    if v_existing_visit is not null then

```

```

if lower(coalesce(v_existing_visit.status, '')) = 'completada' then
    raise exception 'No se puede reprogramar una visita completada';
end if;

update public.technical_visits
set technician_id = p_technician_id,
    scheduled_start = p_scheduled_start,
    scheduled_end = p_scheduled_end,
    status = 'programada'
where id = v_existing_visit.id
returning id into v_visit_id;

v_created := false;
else
insert into public.technical_visits (
    company_id,
    site_survey_id,
    ticket_id,
    installation_project_id,
    technician_id,
    scheduled_start,
    scheduled_end,
    status
)
values (
    p_company_id,
    v_project.survey_id,
    null,
    p_project_id,
    p_technician_id,
    p_scheduled_start,
    p_scheduled_end,
    'programada'
)
on conflict (installation_project_id)
where installation_project_id is not null
do update
    set technician_id = excluded.technician_id,
        scheduled_start = excluded.scheduled_start,
        scheduled_end = excluded.scheduled_end,
        status = 'programada'
returning id into v_visit_id;

v_created := true;
end if;

update public.installation_projects
set responsible_user_id = p_technician_id,
    status = case when status = 'pendiente' then 'en_progreso' else status end
,
    updated_at = now()
where id = p_project_id;

return query select v_visit_id, v_created, 'programada'::text;
end;
$function$
...

```

35. upsert_project_post_installation_check_state

- Schema: public

- Parametros: p_company_id uuid, p_project_id uuid, p_item_id uuid, p_checked boolean, p_notes text, p_user_id uuid
- Clasificacion: RPC / callable
- Archivo separado: database_function_exports/35_upsert_project_post_installation_check_state__p_company_id_uuid_p_project_id_uuid_p_item_id_uuid_p_checked_boolean_p_notes_text_p_user_id_uuid.sql

```
```sql
CREATE OR REPLACE FUNCTION public.upsert_project_post_installation_check_state(p
_company_id uuid, p_project_id uuid, p_item_id uuid, p_checked boolean, p_notes
text DEFAULT NULL::text, p_user_id uuid DEFAULT NULL::uuid)
RETURNS void
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path TO 'public', 'extensions'
AS $function$
declare
 v_uid uuid;
 v_actor uuid;
 v_is_member boolean;
 v_project_exists boolean;
 v_item_exists boolean;
 v_value jsonb;
begin
 v_uid := auth.uid();
 if v_uid is null then
 raise exception 'No autenticado';
 end if;

 v_actor := coalesce(p_user_id, v_uid);

 select exists (
 select 1
 from public.user_roles ur
 where ur.user_id = v_uid
 and ur.company_id = p_company_id
)
 or exists (
 select 1
 from public.user_roles ur
 join public.roles r on r.id = ur.role_id
 where ur.user_id = v_uid
 and ur.company_id is null
 and lower(r.name) = 'admin'
)
 into v_is_member;

 if not coalesce(v_is_member, false) then
 raise exception 'No autorizado para esta compania';
 end if;

 select exists (
 select 1
 from public.installation_projects ip
 where ip.id = p_project_id
 and ip.company_id = p_company_id
)
 into v_project_exists;

 if not v_project_exists then
 raise exception 'Proyecto no encontrado';
 end if;
end if;
```

```

select exists (
 select 1
 from public.post_installation_checks pc
 join public.post_installation_check_items pci on pci.checklist_id = pc.id
 where pc.company_id = p_company_id
 and pci.id = p_item_id
 and pci.is_active = true
)
into v_item_exists;

if not v_item_exists then
 raise exception 'Item de prueba post instalacion no encontrado';
end if;

v_value := jsonb_build_object(
 'checked', p_checked,
 'notes', nullif(btrim(coalesce(p_notes, '')), ''),
 'checked_at', case when p_checked then now() else null end,
 'checked_by', case when p_checked then v_actor else null end
);

update public.installation_projects ip
set post_installation_checks_state = jsonb_set(
 coalesce(ip.post_installation_checks_state, '{}'::jsonb),
 array[p_item_id::text],
 v_value,
 true
),
updated_at = now()
where ip.id = p_project_id
and ip.company_id = p_company_id;
end;
$function$

```